

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

COURSE ASSIGNMENT

Secure Software Delivery for a Healthcare Patient-Service Platform: A Practical DevSecOps Security Lifecycle

Field	Details
Subject Code	CSA5803
Subject Name	DevSecOps Engineering and Application Security
Faculty Name	Dr. R. Pitchandi
Student Name	Lokesh kumar V
Register Number	1925251170
Programme / Semester	B.Tech IT / 2025
Date of Submission	03.09.2026

Problem Statement

A healthcare technology provider maintains a patient-service platform that uses web interfaces, REST APIs and container-based backend services. Frequent software releases depend on third-party packages, automated CI/CD workflows and cloud-style infrastructure. A security review has identified risks such as unsafe source code, vulnerable libraries, exposed credentials, excessive permissions, insecure container settings and limited runtime monitoring.

The engineering objective is to redesign the delivery process so that security is treated as a continuous part of development rather than a final release activity. The proposed solution builds a representative healthcare application, stores it in Git, applies secure review practices, performs automated security testing, enforces release policies, deploys hardened containers and monitors the running service.

The selected laboratory system is a simplified “CareConnect Patient Portal”, implemented as a Python/Flask REST API for patient registration, appointment management and prescription viewing. PostgreSQL is used for persistent storage and a small web interface represents the client layer.

Requirements and Constraints

Category	Requirement / Constraint
Functional	The pipeline shall compile/build, test, scan and package the application. A failed security gate must be followed by remediation and re-validation before release.
Performance	Security checks should normally finish within a few minutes so developers can receive feedback during the normal development cycle.
Technical	The solution should be reproducible on a laptop using freely available/open-source tools.
Cost / Time	The assignment is limited to one representative service and a local Kubernetes environment.
Safety / Security	Credentials must stay outside source control; workload permissions must follow least privilege; serious findings must stop deployment.
Reliability	Malformed and unusually large requests must be rejected safely without application crashes or data loss.
Sustainability	Incremental scans, caching and controlled log retention should be preferred to unnecessary repeated processing.
Ethical	Security experiments are restricted to the student's controlled laboratory environment.
Standards	The design is aligned with OWASP Top 10, OWASP ASVS, NIST SP 800-190, NIST SP 800-61 and CIS Kubernetes guidance.

1. Problem Understanding and Formulation

1.1 Problem Definition

In a modern software delivery environment, a vulnerable commit can move through build, packaging and deployment stages very quickly. For a healthcare application, this creates a significant confidentiality and availability risk because even synthetic patient-service data represents sensitive information. The engineering problem is therefore to establish security checkpoints throughout the source-to-runtime path.

The proposed lifecycle must identify weaknesses early, automatically prevent unacceptable findings from reaching the deployment stage, reduce the privileges available to workloads, provide useful runtime evidence and support a repeatable incident-response process.

1.2 Expected Outcomes

- A working CI/CD workflow that performs testing and multiple security checks before release.
- A documented example of a security failure followed by correction and successful re-validation.
- Container and Kubernetes settings that use non-root execution, restricted privileges and controlled networking.
- A resilience test showing that malformed or oversized requests are handled safely.
- Centralized application logs with detection rules for suspicious authentication and API activity.
- A simulated security incident with detection, containment, remediation and verification evidence.

1.3 Available Information

- The CSA5803 assignment brief and common problem statement.
- Public engineering standards and documentation for OWASP, NIST and CIS Kubernetes practices.
- Open-source tooling documentation for Git, GitHub Actions, Semgrep, Trivy, Gitleaks, OWASP ZAP, Docker and Kubernetes.
- A self-contained healthcare application and local lab environment created for the assignment.

1.4 Assumptions

- The application is a small representative model rather than a complete hospital information system.
- Only synthetic patient records are used during testing.
- A Critical finding or a High finding that violates the defined release policy is considered unacceptable.
- Kubernetes is operated locally using kind or minikube.
- The security tests are performed only against systems owned and controlled for this exercise.

2. Application of Course Knowledge

2.1 Security Principles Applied

- Shift-left security: source-code and dependency checks are executed during pull requests.
- Defence in depth: application, dependency, image, cluster and runtime controls work together.
- Least privilege: CI credentials, database roles and Kubernetes service accounts receive only required permissions.
- Secure-by-default configuration: containers run without unnecessary privileges and filesystems are restricted.
- Infrastructure as code: deployment manifests are version controlled to reduce configuration drift.
- Risk-based prioritisation: severity, likelihood, impact and healthcare-data sensitivity are considered before release.

2.2 Risk Assessment

ID	Risk	Likelihood	Impact	Score	Priority
R1	SQL injection in patient lookup	4	5	20	Critical
R2	Credential/API key stored in source	3	5	15	High
R3	Dependency containing a known CVE	4	4	16	High
R4	Container executed with excessive privileges	3	4	12	High
R5	Over-permissive Kubernetes RBAC	3	4	12	High
R6	Unlimited login attempts	4	4	16	High
R7	Large or malformed request crashes service	3	3	9	Medium
R8	Weak runtime logging and detection	3	4	12	High
R9	Manual deployment changes create configuration drift	2	3	6	Medium

The risk score is calculated as Likelihood $\times$ Impact using a 1–5 scale. The resulting value is used to order remediation work and to support the release-gate decision.

2.3 Incident Response Model

The response process follows Preparation, Detection and Analysis, Containment, Eradication, Recovery and Post-Incident activity. This provides a clear connection between the monitoring system and the engineering process used to correct the underlying weakness.

3. Solution / Design / Methodology

3.1 End-to-End Architecture

The proposed lifecycle consists of source control and peer review, automated verification, container creation, dynamic testing, release policy evaluation, Kubernetes deployment, runtime monitoring and incident response. Each stage produces evidence that can be used by the following stage.

Stage	Purpose	Main Controls / Tools
1. Develop & Review	Reduce insecure changes before merge	Feature branches, pull requests, checklist, Gitleaks hook
2. Build & Test	Verify application behaviour	GitHub Actions, pytest
3. Security Scan	Find source, dependency and secret issues	Semgrep, pip-audit, Gitleaks
4. Package	Create the deployable artifact	Docker multi-stage build, Trivy
5. Dynamic Test	Check the running test service	OWASP ZAP baseline
6. Release Gate	Prevent unacceptable artifacts from release	Policy script and severity thresholds
7. Deploy & Operate	Run securely and reproducibly	Kubernetes, RBAC, NetworkPolicy, non-root pods
8. Monitor & Respond	Detect and recover from events	Fluent Bit, Loki, Grafana, alert rules

3.2 Secure Coding and Review

- Direct commits to the main branch are avoided; changes are submitted through reviewed feature branches.
- Parameterized database queries are required for all user-controlled values.
- Input validation and output encoding are applied at service boundaries.
- Passwords, database URLs, signing keys and API credentials are injected at deployment time rather than committed.
- Database and Kubernetes identities are restricted to the operations they actually need.

3.3 CI/CD Pipeline

The pipeline checks secrets first because secret scanning is comparatively fast, then performs SAST, dependency analysis, tests, image construction, image scanning and a dynamic baseline check. A release-policy script evaluates the resulting JSON reports.

3.4 Example GitHub Actions Workflow

```
name: careconnect-secure-pipeline

on: [pull_request, push]

jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Secret scan
        uses: gitleaks/gitleaks-action@v2

      - name: SAST
        run: semgrep --config p/owasp-top-ten --error --json --output semgrep.json .

      - name: Dependency audit
        run: pip-audit -r requirements.txt -f json -o sca.json

      - name: Application tests
        run: pytest --maxfail=1 -q

      - name: Build container
        run: docker build -t careconnect:${{ github.sha }} .

      - name: Container scan
        run: trivy image --severity HIGH,CRITICAL --format json -o trivy.json careconnect:${{
github.sha }}

      - name: Run test service
        run: docker run -d -p 8080:8080 --name careconnect-test careconnect:${{ github.sha }}

      - name: ZAP baseline
        run: zap-baseline.py -t http://localhost:8080 -J zap.json -I

      - name: Release policy
        run: python ci/check_release.py semgrep.json sca.json trivy.json zap.json
```

3.5 Release-Gate Policy

The release script reads the security reports and returns a non-zero status when a Critical issue or a policy-defined High issue is detected. Because the policy is part of the pipeline, a developer cannot simply ignore a warning and continue to deployment.

3.6 Controlled Vulnerability Demonstration

A controlled SQL injection weakness is introduced into the laboratory patient-search function to demonstrate the effectiveness of the security gate.

Vulnerable version:

```
query = "SELECT * FROM patients WHERE name = '" + patient_name + "'"
cursor.execute(query)
```

The source scanner identifies the unsafe construction and the release policy rejects the build. The code is then changed to a parameterized query:

```
query = "SELECT * FROM patients WHERE name = %s"
cursor.execute(query, (patient_name,))
```

After the correction, the security checks are executed again. The objective is not merely to remove the warning but to demonstrate that the corrected source can complete the same verification path before deployment.

3.7 Container and Kubernetes Security

- Use a small Python base image and a multi-stage build where practical.
- Create and run the application under a dedicated non-root user.
- Use read-only root filesystems where the application permits it.
- Disable privilege escalation and drop unnecessary Linux capabilities.
- Set CPU and memory requests/limits to reduce resource abuse.
- Use a dedicated service account with a narrow RBAC Role.
- Apply a default-deny NetworkPolicy and explicitly allow required traffic.
- Keep database credentials and signing keys in Kubernetes Secrets rather than image layers or Git history.
- Permit deployment only from the image produced by the verified pipeline.

3.8 Resilience / Fuzzing Experiment

A lightweight input-resilience test targets the patient-search endpoint using malformed JSON, long request bodies, special characters, null bytes and random Unicode strings. The purpose is to verify that unexpected input results in controlled errors instead of process termination.

```
payloads = [
    "{",
    "B" * 8000000,
    "' OR '1'='1",
    "\x00\x00",
    random_unicode_string(3000)
]

for payload in payloads:
    response = requests.post(url, data=payload, timeout=5)
    record_result(response.status_code, response.elapsed)
```

The improvement is to apply an application-level request-size limit, an ingress body-size restriction and explicit schema validation. Invalid input should produce HTTP 400 or 413 responses rather than an unhandled exception.

3.9 Centralized Monitoring

Application events are emitted as structured records containing time, endpoint, status and a non-sensitive source identifier. Fluent Bit forwards the records to Loki, while Grafana provides dashboards and alert rules.

Event	Example Detection Rule	Response
Repeated login failures	More than 8 HTTP 401 responses from one IP within 2 minutes	SOC alert and source-IP review
Abnormal request rate	More than 120 requests from one IP within 1 minute	Investigate and rate-limit
Unauthorized admin access	HTTP 403 on /admin/* for a non-admin identity	High-priority alert
Unexpected container process	Process execution outside approved application behaviour	Runtime-security alert

4. Use of Modern Tools

Tool	Category	Purpose
Git / GitHub	Version control	Stores history, feature branches and pull-request reviews.
GitHub Actions	CI/CD	Runs automated build, test, scan and policy stages.
Semgrep	SAST	Checks source code for common security patterns.
Gitleaks	Secret scanning	Detects credentials and tokens accidentally committed to Git.
pip-audit	SCA	Identifies known vulnerabilities in Python dependencies.
Trivy	Container security	Scans filesystem and container image layers for vulnerabilities.
OWASP ZAP	DAST	Performs a baseline assessment against a running test instance.
Docker	Containerisation	Creates a repeatable application artifact.
Kubernetes	Orchestration	Provides declarative deployment, RBAC and network controls.
Fluent Bit + Loki	Logging	Collects and stores structured runtime events.
Grafana	Monitoring	Visualises logs and triggers alert rules.
pytest	Testing	Performs unit and integration verification.
Python requests	Custom testing	Supports controlled resilience and incident simulations.

4.1 Why These Tools Fit the Laboratory

The selected tools cover the major security layers without requiring paid enterprise licences. They also integrate well with a laptop-scale Kubernetes environment. The important design decision is not the number of tools but the way their results are combined into a single release decision.

5. Results and Validation

5.1 Security Results Before and After Correction

Check	Before Remediation	After Remediation	Outcome
SAST	1 Critical SQL injection	0 Critical / High	Resolved
Secret scan	1 High test credential	0 findings	Resolved
SCA	1 High dependency CVE	0 High after upgrade	Resolved
Container scan	2 High base-image findings	0 High after image update	Resolved
DAST	1 Medium header issue	0 Medium after header configuration	Resolved

These values are presented as the target validation record for the alternative assignment. In an actual submission, the student should replace illustrative results with screenshots or exported artifacts from their own lab execution.

5.2 Pipeline Evidence

- A failed pipeline demonstrates that the release policy blocks an unacceptable security finding.
- A remediation commit produces a second pipeline run with the corrected code.
- Security reports and CI logs are retained as evidence for both runs.
- The image deployed to Kubernetes is identified by the source-control commit SHA.

5.3 Resilience Validation

The expected failure mode for an oversized request is a controlled HTTP 413 response. Malformed JSON should result in HTTP 400. Repeating the same test should not increase container restart counts or cause service instability.

5.4 Monitoring Validation

A controlled series of failed authentication attempts is used to verify the complete path from event generation to structured log entry and Grafana alert. The source and time window are preserved so the event can be investigated without relying only on memory.

6. Analysis and Engineering Decisions

6.1 Alternatives Considered

Decision	Option A	Option B	Selected Approach / Reason
CI/CD	GitHub Actions	Jenkins	GitHub Actions; lower setup effort and direct repository integration.
Release policy	Critical only	Critical + High	Critical + High; healthcare information justifies a stricter threshold.
Secrets	Kubernetes Secrets	Vault + External Secrets	Kubernetes Secrets for the lab; Vault is recommended for production.
Logging	Loki + Grafana	ELK	Loki/Grafana for lower laptop resource requirements.
DAST	Baseline scan	Full active scan	Baseline scan for the lab because it is safer and less disruptive.

6.2 Advantages

- Multiple independent controls reduce dependence on one scanner.
- The release gate makes security enforcement repeatable and deterministic.
- Infrastructure as code improves auditability and reduces configuration drift.
- Runtime monitoring provides evidence after deployment, not only during development.
- Non-root and least-privilege controls reduce the impact of container compromise.

6.3 Limitations

- Automated scanners cannot identify every business-logic or design-level weakness.
- The laboratory environment is much smaller than a production healthcare platform.
- Kubernetes Secrets alone are not a complete enterprise secret-management solution.
- A full authenticated DAST assessment and manual penetration test would provide deeper coverage.

6.4 Engineering Trade-offs

Security depth increases verification time. Adding dynamic testing can make a pipeline slower, but the additional confidence is valuable for a sensitive application. Similarly, blocking High findings may create developer friction, so time-limited, security-owner-approved exceptions may be used instead of disabling the gate. A lightweight local logging stack is easier to operate for coursework, while a production deployment would normally require stronger secret management and SIEM capabilities.

6.5 Simulated Incident: Detection, Containment and Recovery

A controlled credential-stuffing simulation is performed against the laboratory login endpoint. A test script generates repeated failed requests from one source IP. The objective is to validate detection engineering and the incident-response workflow.

Relative Time	Event
T+0 min	Controlled login-failure simulation begins.
T+1 min	The authentication-failure threshold is exceeded.
T+1 min	Grafana generates an alert and records the source information.
T+4 min	Analyst reviews centralized logs and confirms repeated attempts.
T+6 min	The source is blocked at the ingress/network layer and active sessions are reviewed.
T+12 min	Root cause is identified as insufficient login rate limiting.
T+25 min	Rate limiting and account lockout controls are added and tested.
T+32 min	Corrected image is deployed through the normal CI/CD process.
T+38 min	The simulation is repeated and excessive requests receive HTTP 429 responses.

The important outcome is that the fix is delivered through the same controlled pipeline rather than by an undocumented manual change. This preserves traceability from source change to deployment and monitoring evidence.

7. Broader Considerations

7.1 Safety and Privacy

Healthcare software must protect confidentiality, integrity and availability. Least-privilege access, secret protection, input validation and audit logging reduce the chance that a defect becomes a serious data-security event.

7.2 Ethics and Professional Responsibility

All vulnerability demonstrations and attack simulations in this assignment are intended for owned laboratory systems. Security testing against real healthcare systems would require explicit authorization, controlled scope and responsible handling of findings.

7.3 Economic Considerations

Open-source tools reduce licensing expenditure, making the design suitable for academic and small-team environments. The main investment is engineering time. Preventing a serious security incident can avoid substantially higher costs associated with downtime, investigation, remediation and loss of trust.

7.4 Sustainability

The laboratory can reduce resource consumption by caching dependencies, running lightweight checks on every change and reserving heavier scans for scheduled or release-oriented jobs. Log retention should also be limited to the period needed for operational and security analysis.

7.5 Accessibility and Societal Impact

Security dashboards and alert messages should be understandable to on-call engineers who may not be security specialists. Reliable patient-service applications also have a social impact because service outages or security incidents can affect access to healthcare-related services.

8. Conclusion

This assignment presents a connected DevSecOps lifecycle for a representative healthcare patient-service platform. The design integrates secure coding, pull-request review, secret scanning, static analysis, dependency auditing, container scanning, dynamic testing and an automated release gate.

The deployment model strengthens the runtime environment through non-root containers, restricted capabilities, Kubernetes RBAC, NetworkPolicy and externalised secrets. Resilience testing adds another layer by checking how the application behaves when it receives malformed or unusually large input.

Centralized logging and alerting extend security beyond the build stage. A simulated authentication attack demonstrates how suspicious activity can be detected, investigated, contained and remediated while preserving an auditable connection between the source repository, CI pipeline, deployment and monitoring systems.

The overall lesson is that DevSecOps is not simply the installation of security tools. Its value comes from integrating security decisions into the normal engineering workflow so that secure behaviour becomes repeatable, measurable and recoverable.

9. Student Reflection

9.1 Learning Beyond the Basic Concepts

- I understood that effective DevSecOps depends on integrating several security checks rather than relying on one scanner.
- I learned that a release gate turns security requirements into an enforceable engineering rule.
- The controlled input test showed how application reliability and security are closely connected.
- Monitoring thresholds require careful tuning because overly sensitive rules can generate excessive noise.
- Version-controlled deployment files improve both reproducibility and security accountability.

9.2 Improvements With More Time

- Create a formal STRIDE threat model for the complete patient-service platform.
- Introduce Vault or an External Secrets solution for stronger secret management.
- Add image signing and verification for supply-chain assurance.
- Perform authenticated DAST and business-logic testing.
- Schedule recurring scans for newly disclosed dependency and image vulnerabilities.
- Conduct a broader controlled security exercise to evaluate monitoring coverage.

10. References

1. OWASP Foundation. OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks.
2. OWASP Foundation. OWASP Application Security Verification Standard (ASVS).
3. National Institute of Standards and Technology. SP 800-190: Application Container Security Guide.
4. National Institute of Standards and Technology. SP 800-61 Rev. 2: Computer Security Incident Handling Guide.
5. Center for Internet Security. CIS Kubernetes Benchmark.
6. Semgrep Documentation.
7. Trivy Documentation.
8. Gitleaks Documentation.
9. OWASP ZAP Documentation.
10. Kubernetes Documentation – Pod Security Standards, RBAC and Network Policies.
11. GitHub Actions Documentation.
12. Docker Documentation – Dockerfile Best Practices.
13. MITRE ATT&CK – Brute Force / Credential Access.
14. Grafana and Loki Documentation.